

PRÁCTICA N° 2

Técnicas de Búsqueda basadas en Poblaciones

Problema de Asignación Con Restricciones (PAR)

Nombre: Alejandra Peñaherrera Díaz

NIE: Y8676958S

E-mail: alepenaherrera@correo.ugr.es

Asignatura: Metaheurísticas

Curso Académico: 2025 – 2026

Horario de Prácticas: Viernes 15:30-17:30 (A2)

17 de mayo de 2026

Índice

1. Descripción del Problema	1
2. Aplicación de los Algoritmos al Problema	2
2.1. Representación de la solución	2
2.2. Función Objetivo	2
2.3. Operadores Comunes	3
2.3.1. Generación de solución aleatoria válida	3
2.3.2. Reparación de solución	4
2.3.3. Validación de solución	4
2.3.4. Operador de selección por torneo	5
2.3.5. Operador de cruce uniforme	5
2.3.6. Operador de cruce por segmento fijo	6
2.3.7. Operador de mutación uniforme	6
3. Descripción de los Algoritmos	6
3.1. Algoritmo Greedy COPKM	6
3.2. Algoritmo Aleatorio (Random Search)	9
3.3. Búsqueda Local (BL)	9
3.4. Algoritmo Genético Generacional con Elitismo (AGG)	11
3.5. Algoritmo Genético Estacionario (AGE)	12
3.6. Algoritmos Meméticos (AM)	13
3.6.1. Búsqueda Local Suave (BLS)	14
3.6.2. Variantes del Algoritmo Memético	14
4. Estructura del Código y Manual de Usuario	15
4.1. Estructura de Directorios	15
4.2. Compilación	16
4.3. Ejecución	16
4.4. Ejemplos de Ejecución	16
5. Experimentos y Análisis de Resultados	17
5.1. Configuración Experimental	17
5.2. Resultados Obtenidos	17
5.3. Análisis de Resultados	19
5.3.1. Greedy y Random como referencia	19
5.3.2. Búsqueda Local como el algoritmo más fuerte	20
5.3.3. AGG vs AGE: influencia del esquema de evolución	20
5.3.4. AGG-UN vs AGG-SF y AGE-UN vs AGE-SF: influencia del operador de cruce	21
5.3.5. Algoritmos Meméticos: balance exploración-explotación	21
5.3.6. Efecto del nivel de restricciones	22
5.3.7. Conclusiones	22

1. Descripción del Problema

El Problema de Asignación Con Restricciones (PAR) es una generalización del problema clásico de agrupamiento o clustering. A diferencia del clustering tradicional, que únicamente considera la distancia entre instancias, el PAR incorpora un nuevo tipo de información en forma de restricciones que imponen relaciones específicas entre parejas de instancias durante el proceso de agrupamiento.

Dado un conjunto de datos X con n instancias, cada una caracterizada por d atributos numéricos, el objetivo del PAR es encontrar una partición $C = \{c_0, c_1, \dots, c_{k-1}\}$ que asigne cada instancia a exactamente uno de los k clusters, de tal manera que se minimice simultáneamente la distancia intra-cluster (la cohesión de los grupos formados) y el incumplimiento de restricciones (la violación de las relaciones especificadas entre instancias). El problema considera dos tipos de restricciones binarias sobre pares de instancias:

- Must-Link (ML): indica que dos instancias deben pertenecer al mismo cluster
- Cannot-Link (CL): indica que dos instancias no pueden pertenecer al mismo cluster

El objetivo es obtener una partición solución que permita minimizar la función fitness definida como:

$$\text{fitness}(\mathcal{C}) = \text{desviación}(\mathcal{C}) + \lambda \cdot \text{infeasibility}(\mathcal{C}) \quad (1)$$

Para calcular dicha función objetivo, es necesario primer definir ciertos conceptos. Definimos el centroide $\vec{\mu}_i$ asociado a cada grupo c_i de una partición C como el vector promedio de las instancias de X que lo componen:

$$\vec{\mu}_i = \frac{1}{|c_i|} \sum_{\vec{x}_j \in c_i} \vec{x}_j \quad (2)$$

Definimos la distancia media intra-cluster de un cluster C_i como la media de las distancias, de las instancias que lo conforman, a su centroide:

$$\text{distancia_intra}(c_i) = \frac{1}{|c_i|} \sum_{\vec{x}_j \in c_i} \|\vec{x}_j - \vec{\mu}_i\| \quad (3)$$

Definimos la desviación como la distancia media intra-cluster:

$$\text{desviación}(\mathcal{C}) = \frac{1}{k} \sum_{c_i \in \mathcal{C}} \text{distancia_intra}(c_i) \quad (4)$$

Definimos la infeasibility como el número total de restricciones incumplidas por una partición C :

$$\text{infeasibility}(\mathcal{C}) = \sum_{(i,j) \in \text{ML}} \mathbb{I}[c(x_i) \neq c(x_j)] + \sum_{(i,j) \in \text{CL}} \mathbb{I}[c(x_i) = c(x_j)] \quad (5)$$

Por último, λ se define como el parámetro de ponderación que equilibra ambos objetivos y se calcula como:

$$\lambda = \frac{\text{máx dist}(\mathcal{X})}{\mathcal{R}} \quad (6)$$

donde $\mathcal{R}$ es el número total de restricciones y $\text{máx dist}(\mathcal{X})$ es la distancia máxima entre cualquier par de instancias del conjunto de datos.

2. Aplicación de los Algoritmos al Problema

2.1. Representación de la solución

Una solución se representa mediante un vector entero $S = (s_0, s_1, \dots, s_{n-1})$ de tamaño n , donde cada s_i contiene un valor en $\{0, 1, \dots, k-1\}$ que indica el cluster al que pertenece la instancia i . siendo requisito que cada cluster contenga al menos una instancia. Una solución es válida si y sólo si cada cluster tiene al menos una instancia asignada. Para evaluar la calidad de una solución se utilizan métricas como Fitness (valor de la función objetivo), Distancia (promedio de la distancia intra-cluster), Incumplimientos (proporción de restricciones violadas), Evaluaciones (número total de evaluaciones realizadas) y Tiempo (tiempo de ejecución en segundos).

2.2. Función Objetivo

Algorithm 1 Función objetivo – $\text{fitness}(S)$

Require: Solución S , dataset $\mathcal{X}$, listas ML, CL, λ

Ensure: Valor de fitness

- 1: $\text{dev} \leftarrow \text{deviation}(S)$
 - 2: $\text{inf} \leftarrow \text{infeasibility}(S)$
 - 3: **return** $\text{dev} + \lambda \cdot \text{inf}$
-

Algorithm 2 Función objetivo – $\text{fitness}(S, \text{pos}, \text{nuevoVal})$

Require: Solución S , posición pos de la solución a modificar, nuevo valor nuevoVal

Ensure: Fitness de la solución vecina

- 1: $S' \leftarrow$ copia de S
 - 2: $S'[\text{pos}] \leftarrow \text{nuevoVal}$
 - 3: **return** $\text{fitness}(S')$
-

Algorithm 3 Infeasibility – $\text{infeasibility}(S)$

Require: Solución S , lista ML, lista CL

Ensure: Número de restricciones incumplidas

- 1: $\text{inf} \leftarrow 0$
 - 2: **para cada** $(i, j) \in \text{ML}$ **hacer**
 - 3: **if** $S[i] \neq S[j]$ **then** $\text{inf} \leftarrow \text{inf} + 1$
 - 4: **para cada** $(i, j) \in \text{CL}$ **hacer**
 - 5: **if** $S[i] = S[j]$ **then** $\text{inf} \leftarrow \text{inf} + 1$
 - 6: **return** inf
-

Algorithm 4 Desviación – `deviation`(S)**Require:** Solución S , dataset $\mathcal{X}$, k , n , d **Ensure:** Desviación general de la partición

```

1: centroides[0.. $k-1$ ]  $\leftarrow \vec{0}$ ,  tamaños[0.. $k-1$ ]  $\leftarrow 0$ 
2: for  $i \leftarrow 0$  hasta  $n-1$  do
3:    $c \leftarrow S[i]$ ;  tamaños[ $c$ ] += 1
4:   for  $j \leftarrow 0$  hasta  $d-1$  do
5:     centroides[ $c$ ][ $j$ ] +=  $\mathcal{X}[i][j]$ 
6: for  $c \leftarrow 0$  hasta  $k-1$  do
7:   if tamaños[ $c$ ] > 0 then
8:     for  $j \leftarrow 0$  hasta  $d-1$  do
9:       centroides[ $c$ ][ $j$ ] /= tamaños[ $c$ ]
10: desviación  $\leftarrow 0$ 
11: for  $c \leftarrow 0$  hasta  $k-1$  do
12:   intra  $\leftarrow 0$ 
13:   for  $i \leftarrow 0$  hasta  $n-1$  do
14:     if  $S[i] = c$  then
15:       intra +=  $\|\mathcal{X}[i] - \text{centroides}[c]\|_2$ 
16:   if tamaños[ $c$ ] > 0 then
17:     intra /= tamaños[ $c$ ]
18:   desviación += intra
19: return desviación /  $k$ 

```

El parámetro λ presente en la función fitness pondera el término de infeasibility en la función objetivo y se calcula una única vez al cargar el problema:

$$\lambda = \frac{\max_{i,j} \|\vec{x}_i - \vec{x}_j\|_2}{|\mathcal{R}|} \quad (7)$$

donde $|\mathcal{R}| = |\text{ML}| + |\text{CL}|$ es el número total de restricciones y el numerador es la distancia euclídea máxima entre cualquier par de instancias del conjunto de datos. Si no hay restricciones, $\lambda = 0$.

2.3. Operadores Comunes

2.3.1. Generación de solución aleatoria válida

Permite construir su solución inicial. Garantiza que ningún cluster quede vacío asignando forzosamente un cluster distinto a los primeros k índices barajados antes de repartir el resto aleatoriamente.

Algorithm 5 Generar Solución Inicial Aleatoria – `createSolution()`

Require: n instancias, k clusters**Ensure:** Vector S válido con al menos una instancia por cluster

```

1:  $S \leftarrow$  vector de enteros de tamaño  $n$ 
2:  $idxs \leftarrow [0, 1, 2, \dots, n - 1]$ 
3:  $ORDENALEATORIO(idxs)$ 
4: for  $c \leftarrow 0$  hasta  $k - 1$  do
5:    $S[idxs[c]] \leftarrow c$ 
6: for  $i \leftarrow k$  hasta  $n - 1$  do
7:    $S[idxs[i]] \leftarrow ENTEROALEATORIO(0, k - 1)$ 
8: return  $S$ 

```

2.3.2. Reparación de solución

Cuando una operación produce una solución en la que algún cluster queda vacío, se aplica el operador `fix` para restaurar su validez. Localiza el cluster más poblado, extrae aleatoriamente una de sus instancias y la reasigna al cluster vacío, repitiendo el proceso hasta que todos los clusters tengan al menos un elemento.

Algorithm 6 Reparación – `fix( $S$ )`

Require: Solución S posiblemente inválida, k clusters, n instancias**Ensure:** S con al menos una instancia por cluster

```

1: for  $c \leftarrow 0$  hasta  $k - 1$  do
2:    $clusters[c] \leftarrow \{i : S[i] = c\}$ 
3: for  $c \leftarrow 0$  hasta  $k - 1$  do
4:   if  $clusters[c] = \emptyset$  then
5:      $src \leftarrow 0$ 
6:     for  $c' \leftarrow 1$  hasta  $k - 1$  do
7:       if  $|clusters[c']| > |clusters[src]|$  then
8:          $src \leftarrow c'$ 
9:      $inst \leftarrow$  elemento en posición aleatoria de  $clusters[src]$ 
10:    Eliminar  $inst$  de  $clusters[src]$ 
11:     $clusters[c] \leftarrow clusters[c] \cup \{inst\}$ 
12:     $S[inst] \leftarrow c$ 
13: return  $S$ 

```

2.3.3. Validación de solución

Una solución S es válida si y sólo si cada cluster $c \in \{0, \dots, k - 1\}$ tiene al menos una instancia asignada. Para comprobarlo, se marca cada cluster que aparece en S y se verifica que todos hayan sido marcados.

Algorithm 7 Validación – isValid(S)**Require:** Solución S , número de clusters k **Ensure:** verdadero si todos los clusters tienen al menos una instancia, falso en caso contrario

```

1: visto[0.. $k - 1$ ]  $\leftarrow$  falso
2: para cada valor  $v \in S$  hacer
3:   visto[ $v$ ]  $\leftarrow$  verdadero
4: for cluster  $c \leftarrow 0$  hasta  $k - 1$  do
5:   if visto[ $c$ ] = falso then
6:     return falso
7: return verdadero

```

2.3.4. Operador de selección por torneo

Para seleccionar un padre se escogen aleatoriamente 3 individuos de la población y se devuelve el de menor fitness. En el esquema generacional se realizan M torneos para construir la población de padres, es decir, tantos torneos como individuos existan. En el estacionario se realizan 2 torneos.

Algorithm 8 Selección por torneo – TOURNAMENTSELECT(pop, fits)**Require:** Población pop, vector de fitness fits, tamaño de torneo = 3**Ensure:** Índice del individuo ganador

```

1:  $a, b, c \leftarrow$  índices aleatorios distintos en  $\{0, \dots, M - 1\}$ 
2: mejor  $\leftarrow a$ 
3: if fits[ $b$ ] < fits[mejor] then mejor  $\leftarrow b$ 
4: if fits[ $c$ ] < fits[mejor] then mejor  $\leftarrow c$ 
5: return mejor

```

2.3.5. Operador de cruce uniforme

Para cada posición i del cromosoma, se genera un número aleatorio $r \in [0, 1]$. Si $r < 0,5$ el hijo 1 hereda de P1 y el hijo 2 de P2; en caso contrario se intercambian. Formalmente:

$$\text{Off}_i^1, \text{Off}_i^2 = \begin{cases} S_i^1, S_i^2 & \text{si rand() < 0,5} \\ S_i^2, S_i^1 & \text{en otro caso} \end{cases} \quad (8)$$

Algorithm 9 Cruce Uniforme – CROSSOVERUNIFORM(p1, p2)**Require:** Padres p1, p2 de longitud n **Ensure:** Par de hijos (c1, c2)

```

1: Inicializar c1, c2 de longitud  $n$ 
2: for  $i \leftarrow 0$  hasta  $n - 1$  do
3:   if RealAleatorio(0, 1) < 0,5 then
4:     c1[ $i$ ]  $\leftarrow$  p1[ $i$ ]; c2[ $i$ ]  $\leftarrow$  p2[ $i$ ]
5:   else
6:     c1[ $i$ ]  $\leftarrow$  p2[ $i$ ]; c2[ $i$ ]  $\leftarrow$  p1[ $i$ ]
7: return (c1, c2)

```

2.3.6. Operador de cruce por segmento fijo

Se genera una posición inicial aleatoria $pos \in [0, N - 1]$ y un tamaño de segmento $FN \in [1, N/3]$. El hijo 1 hereda el segmento de P1 y el hijo 2 de P2; las posiciones fuera del segmento se rellenan aplicando cruce uniforme entre ambos padres. El segmento puede solapar el final del vector por lo que se aplica módulo N .

Algorithm 10 Cruce por Segmento Fijo – CROSOVERFIXEDSEGMENT(p1, p2)

Require: Padres p1, p2 de longitud n

Ensure: Par de hijos (c1, c2)

```

1: inPos  $\leftarrow$  EnteroAleatorio(0,  $n - 1$ )
2: segSize  $\leftarrow$  EnteroAleatorio(1,  $\lfloor n/3 \rfloor$ )
3: endPos  $\leftarrow$  inPos + segSize - 1
4: Marcar como segmento las posiciones  $\{(inPos + s) \bmod n : 0 \leq s < segSize\}$ 
5: (fill1, fill2)  $\leftarrow$  crossoverUniform(p1, p2)
6: for  $i \leftarrow 0$  hasta  $n - 1$  do
7:   if  $i \in$  segmento then
8:      $c1[i] \leftarrow p1[i]; \quad c2[i] \leftarrow p2[i]$ 
9:   else
10:     $c1[i] \leftarrow fill1[i]; \quad c2[i] \leftarrow fill2[i]$ 
11: return (c1, c2)
```

2.3.7. Operador de mutación uniforme

Se selecciona aleatoriamente un gen (posición $i \in \{0, \dots, n - 1\}$) y se le asigna un nuevo valor de cluster $l \in \{0, \dots, k - 1\}$, $l \neq S[i]$, verificando que el cluster de origen no quede vacío.

Algorithm 11 Mutación Uniforme – MUTATE(sol, k)

Require: Solución sol, número de clusters k

```

1: gen  $\leftarrow$  EnteroAleatorio(0,  $n - 1$ )
2: oldVal  $\leftarrow$  sol[gen]
3: count  $\leftarrow |\{j : sol[j] = oldVal\}|$ 
4: if count  $\leq 1$  then return ▷ No mutar: dejaría cluster vacío
5: newVal  $\leftarrow$  EnteroAleatorio(0,  $k - 2$ )
6: if newVal  $\geq$  oldVal then newVal  $\leftarrow$  newVal + 1
7: sol[gen]  $\leftarrow$  newVal
```

3. Descripción de los Algoritmos

3.1. Algoritmo Greedy COPKM

El algoritmo COPKM es una adaptación del algoritmo k -medias clásico que incorpora restricciones de tipo *Must-Link* (ML) y *Cannot-Link* (CL) mediante una interpretación débil de las mismas; es decir, en lugar de rechazar asignaciones que las violen, las penaliza a través de una función heurística. El algoritmo comienza generando k centroides iniciales aleatorios dentro del rango de cada atributo del conjunto de datos. A continuación,

itera barajando aleatoriamente las instancias en cada paso y asignando cada instancia al cluster que minimiza simultáneamente el número de restricciones violadas y la distancia al centroide. Los centroides se actualizan tras cada iteración completa y el proceso continúa hasta que la solución no cambia o se alcanza el número máximo de iteraciones (100 en este caso).

La función heurística utilizada para asignar cada instancia x_i a un cluster c evalúa dos criterios en orden de prioridad: en primer lugar, el incremento en *infeasibility* que produciría dicha asignación, calculado como el número de restricciones ML y CL que se violarían teniendo en cuenta únicamente las instancias ya asignadas; en segundo lugar, en caso de empate, la distancia Euclídea entre la instancia x_i y el centroide μ_c del cluster. De este modo, se prioriza siempre la satisfacción de restricciones sobre la minimización de la distancia intra-cluster.

Pseudocódigo

Algorithm 12 Calcular Centroides – `computeCentroids(problema, S)`

Require: Solución S , dataset $\mathcal{X}$, k clusters, n instancias, d atributos

Ensure: $\vec{\mu}[0..k-1]$: vector de centroides

```

1:  $\mu \leftarrow$  vector de  $[0, k-1]$  instancias inicializadas a 0 de tamaño  $d$ 
2:  $sizes[0..k-1] \leftarrow 0$  ▷ Número de instancias por cluster
3: for  $i \leftarrow 0$  hasta  $n-1$  do
4:    $c \leftarrow S[i]$ 
5:    $sizes[c] \leftarrow sizes[c] + 1$ 
6:   for  $j \leftarrow 0$  hasta  $a-1$  do
7:      $\vec{\mu}[c][j] \leftarrow \vec{\mu}[c][j] + X[i][j]$ 
8: for  $c \leftarrow 1$  hasta  $k$  do
9:   if  $sizes[c] > 0$  then
10:    for  $j \leftarrow 0$  hasta  $d-1$  do
11:       $\vec{\mu}[c][j] \leftarrow \vec{\mu}[c][j] / sizes[c]$ 
12: return  $\vec{\mu}$ 

```

Algorithm 13 COPKM Greedy – optimize(problema, maxEvals)

Require: Problema PAR con n instancias, k clusters, d atributos por instancia, dataset X y conjunto de restricciones R

Ensure: Solución S , su fitness y número de evaluaciones

```

1:  $dmin[j] \leftarrow \min_{x \in X} x[j] \quad \forall j \in \{0, \dots, a-1\}$ 
2:  $dmax[j] \leftarrow \max_{x \in X} x[j] \quad \forall j \in \{0, \dots, a-1\}$ 
3: for  $c \leftarrow 0$  hasta  $k-1$  do
4:   for  $j \leftarrow 0$  hasta  $d-1$  do
5:      $\vec{\mu}[c][j] \leftarrow \text{REALALEATORIO}(dmin[j], dmax[j])$ 
6:  $S \leftarrow$  vector de  $n$  ceros
7:  $changed \leftarrow \text{verdadero}$ 
8:  $iter \leftarrow 0$ 
9:  $idxs \leftarrow [0, 1, \dots, n-1]$ 
10: while  $changed$  y  $iter < maxevals$  do
11:    $changed \leftarrow \text{falso}$ 
12:    $iter \leftarrow iter + 1$ 
13:    $\text{ORDENALEATORIO}(idxs)$ 
14:    $S' \leftarrow$  vector de  $n$  enteros
15:   for cada  $idx$  en  $idxs$  do
16:      $mejor\_cluster \leftarrow 0$ 
17:      $mejor\_inf \leftarrow \infty$ 
18:      $mejor\_dist \leftarrow \infty$ 
19:     for  $c \leftarrow 0$  hasta  $k-1$  do
20:        $dist \leftarrow \text{EUCLIDEA}(X[idx], \vec{\mu}[c])$ 
21:        $inf \leftarrow 0$ 
22:       for cada  $(i, j) \in ML$  do
23:         if  $i = idx$  y  $S[j] \neq c$  then  $inf \leftarrow inf + 1$ 
24:         if  $j = idx$  y  $S[i] \neq c$  then  $inf \leftarrow inf + 1$ 
25:       for cada  $(i, j) \in CL$  do
26:         if  $i = idx$  y  $S[j] = c$  then  $inf \leftarrow inf + 1$ 
27:         if  $j = idx$  y  $S[i] = c$  then  $inf \leftarrow inf + 1$ 
28:       if  $inf < mejor\_inf$  o ( $inf = mejor\_inf$  y  $dist < mejor\_dist$ ) then
29:          $mejor\_inf \leftarrow inf$ 
30:          $mejor\_dist \leftarrow dist$ 
31:          $mejor\_cluster \leftarrow c$ 
32:        $S'[idx] \leftarrow mejor\_cluster$ 
33:   if no  $\text{ISVALID}(S')$  then
34:      $\text{FIX}(S')$ 
35:   if  $S' \neq S$  then
36:      $changed \leftarrow \text{verdadero}$ 
37:      $S \leftarrow S'$ 
38:      $\vec{\mu} \leftarrow \text{COMPUTECENTROIDS}(S)$ 
39: return  $(S, \text{fitness}(S), 1)$ 

```

3.2. Algoritmo Aleatorio (Random Search)

Genera soluciones completamente aleatorias de forma reiterada hasta consumir el presupuesto de evaluaciones, devolviendo la mejor encontrada. Sirve como cota inferior de referencia para valorar la mejora aportada por los algoritmos más elaborados.

Pseudocódigo

Algorithm 14 Búsqueda Aleatoria – `optimize(problema, maxEvals)`

Require: Problema PAR, `maxEvals` = 100 000

Ensure: Mejor solución encontrada S^* y su fitness f^*

```

1: Variables:
2:    $S^*$ : mejor solución encontrada hasta el momento
3:    $f^*$ : fitness de  $S^*$ 
4:    $S$ : solución aleatoria generada en la iteración actual
5:    $f$ : fitness de  $S$ 
6:    $S^* \leftarrow \emptyset$ ,  $f^* \leftarrow +\infty$ 
7: for evaluación  $\leftarrow 1$  hasta maxEvals do
8:    $S \leftarrow \text{crearSolución}()$ 
9:    $f \leftarrow \text{fitness}(S)$ 
10:  if  $f < f^*$  then
11:     $f^* \leftarrow f$ ,  $S^* \leftarrow S$ 
12: return ( $S^*$ ,  $f^*$ , maxEvals)

```

3.3. Búsqueda Local (BL)

La Búsqueda Local implementa el esquema *primer mejor* (*first improvement*): explora el entorno de la solución actual en orden aleatorio y acepta el primer vecino que estrictamente mejore el fitness. Tras cada mejora reconstruye el entorno completo con la nueva asignación. Se detiene al no encontrar mejora en todo el entorno o al alcanzar las 100 000 evaluaciones.

Generación de la solución inicial

La solución inicial se genera mediante `createSolution()`, descrito en la Sección 2.3, que produce una asignación aleatoria válida con valores en $\{0, \dots, k-1\}$ garantizando que ningún cluster queda vacío. Esta solución se evalúa y cuenta como la primera evaluación de la función objetivo.

Operador de generación de vecino

Un vecino de la solución actual S se obtiene tomando una instancia $inst \in [0, k)$ y cambiando su cluster asignado $S[inst]$ por un cluster destino diferente $nuevoC \in [0, k)$, $nuevoC \neq S[inst]$. El resto de asignaciones del vector S permanecen inalteradas. El vecino generado por el par $(inst, nuevoC)$ es por tanto:

$$S'[i] = \begin{cases} nuevoC & \text{si } i = inst \\ S[i] & \text{en otro caso} \end{cases} \quad (9)$$

El movimiento sólo se considera válido si el cluster de origen $S[inst]$ contiene más de una instancia, garantizando que ningún cluster quede vacío tras el movimiento sin necesidad de llamar a `fix`

Exploración del entorno

El entorno $\mathcal{N}$ se representa como la lista de pares $(inst, offset)$ con $inst \in [0, n)$ y $offset \in [1, k)$, de tamaño $n(k - 1)$. El cluster destino se calcula como $nuevoC = (S[inst] + offset) \bmod k$, lo que permite que el entorno permanezca invariante aunque la solución cambie: al actualizar $S[inst]$, los pares afectados apuntan automáticamente a nuevos clusters destino sin necesidad de reconstruir el vecindario tras cada mejora. En cada iteración del bucle externo se baraja $\mathcal{N}$ completamente antes de recorrerlo, garantizando un orden de exploración aleatorio sin repetición.

Para verificar en $O(1)$ si un movimiento dejaría algún cluster vacío, se mantiene un vector auxiliar `clusterSize` de tamaño k inicializado con el número de instancias asignadas a cada cluster. Tras aceptar una mejora, `clusterSize` se actualiza decrementando el cluster de origen e incrementando el cluster destino, evitando así recorrer la solución completa en cada comprobación.

Factorización de la función objetivo

La implementación no factoriza incrementalmente el fitness, cada llamada a `fitness(S, inst, nuevoC)` genera internamente una copia S' con $S'[inst] = nuevoC$ y evalúa la función objetivo completa desde cero, sin reutilizar información de evaluaciones anteriores.

Pseudocódigo

Algorithm 15 Búsqueda Local – `optimize(problema, maxEvals)`

Require: Problema PAR con n instancias y k clusters, $maxevals$

Ensure: $S, f, evals$

```

1:  $S \leftarrow \text{CREATE\_SOLUTION}()$ 
2:  $f \leftarrow \text{FITNESS}(S)$ 
3:  $evals \leftarrow 1$ 
4:  $clusterSize[c] \leftarrow |\{j : S[j] = c\}| \quad \forall c \in [0, k)$ 
5:  $vecindario \leftarrow \emptyset$ 
6: for  $i \leftarrow 0$  hasta  $n - 1$  do
7:   for  $offset \leftarrow 1$  hasta  $k - 1$  do
8:      $vecindario.AÑADIR(i, offset)$ 
9:  $mejora \leftarrow \text{verdadero}$ 
10: while  $mejora$  y  $evals < maxevals$  do
11:    $mejora \leftarrow \text{falso}$ 
12:    $ORDENALEATORIO(vecindario)$ 
13:   for cada  $(i, offset)$  en  $vecindario$  do
14:     if  $evals \geq maxevals$  then break
15:      $clusterOrigen \leftarrow S[i]$ 
16:      $clusterDestino \leftarrow (clusterOrigen + offset) \bmod k$ 
17:     if  $clusterSize[clusterOrigen] \leq 1$  then continuar
18:      $f' \leftarrow \text{FITNESS}(S, i, clusterDestino)$ 
19:      $evals \leftarrow evals + 1$ 
20:     if  $f' < f$  then
21:        $S[i] \leftarrow clusterDestino$ 
22:        $clusterSize[clusterOrigen] -= 1$ 
23:        $clusterSize[clusterDestino] += 1$ 
24:        $f \leftarrow f'$ 
25:        $mejora \leftarrow \text{verdadero}$ 
26:       break
27: return  $S, f, evals$ 

```

3.4. Algoritmo Genético Generacional con Elitismo (AGG)

El AGG reemplaza la población completa en cada generación. Tras reemplazar, se aplica elitismo: si el mejor individuo de la nueva generación es peor que el mejor de la generación anterior, se sustituye el peor hijo por dicho mejor previo. Se implementan además dos variantes según el operador de cruce: **AGG-UN** (cruce uniforme) y **AGG-SF** (segmento fijo).

Para este algoritmo la población será $M = 50$, la probabilidad de cruce será de $P_c = 0,8$, la probabilidad de mutación será de $P_m = 0,1$, y el número máximo de evaluaciones de la función objetivo será de $maxevals = 100000$.

Algorithm 16 AGG – OPTIMIZE(par, maxevals)**Require:** Problema PAR, $M = 50$, $P_c = 0,8$, $P_m = 0,1$, maxevals**Ensure:** Mejor solución encontrada y su fitness

```

1: for  $i \leftarrow 0$  hasta  $M - 1$  do
2:   pop[i]  $\leftarrow$  CREATESOLUTION()
3:   fits[i]  $\leftarrow$  FITNESS(pop[i]); evals += 1
4: (bestSol, bestFit)  $\leftarrow$  mejor individuo de pop
5:
6: while evals < maxevals do
7:   for  $i \leftarrow 0$  hasta  $M - 1$  do                                ▷ Selección
8:     parents[i]  $\leftarrow$  pop[TOURNAMENTSELECT(pop, fits)]
9:
10:  numCrosses  $\leftarrow$  round( $P_c \cdot M/2$ )                                ▷ Cruce
11:  for  $p \leftarrow 0$  hasta numCrosses – 1 do
12:    ( $c1, c2$ )  $\leftarrow$  DOCROSSOVER(parents[2p], parents[2p + 1])
13:    offspring[2p]  $\leftarrow$  c1; offspring[2p + 1]  $\leftarrow$  c2
14:  for  $j \leftarrow$  numCrosses  $\cdot$  2 hasta  $M - 1$  do
15:    offspring[j]  $\leftarrow$  parents[j]
16:
17:  for  $m \leftarrow 0$  hasta round( $P_m \cdot M$ ) – 1 do                                ▷ Mutación
18:    MUTATE(offspring[EnteroAleatorio(0, M – 1)], k)
19:
20:  for cada  $s$  en offspring do                                ▷ Reparación
21:    if  $\neg$ ISVALID( $s$ ) then FIX( $s$ )
22:
23:  newFits[0.. $M - 1$ ]  $\leftarrow$   $+\infty$                                 ▷ Evaluación
24:  for  $i \leftarrow 0$  hasta  $M - 1$  mientras evals < maxevals do
25:    newFits[i]  $\leftarrow$  FITNESS(offspring[i]); evals += 1
26:
27:  if bestFit < mín(newFits) then                                ▷ Elitismo
28:    offspring[arg máx(newFits)]  $\leftarrow$  bestSol
29:    newFits[arg máx(newFits)]  $\leftarrow$  bestFit
30:
31:  pop  $\leftarrow$  offspring; fits  $\leftarrow$  newFits                                ▷ Reemplazo
32:  if mín(fits) < bestFit then
33:    (bestSol, bestFit)  $\leftarrow$  mejor individuo de pop
34: return bestSol, bestFit

```

3.5. Algoritmo Genético Estacionario (AGE)

El AGE genera únicamente 2 hijos por iteración. Los hijos compiten con los 2 peores individuos de la población, y un hijo reemplaza al peor solo si lo supera. Se implementan asimismo **AGE-UN** y **AGE-SF**.

Para este algoritmo la población será $M = 50$, la probabilidad de cruce será de $P_c = 1$, pues siempre se cruza, la probabilidad de mutación será de $P_m = 0,1$, y el número máximo de evaluaciones de la función objetivo será de $maxevals = 100000$.

Algorithm 17 AGE – OPTIMIZE(par, maxevals)**Require:** Problema PAR, $M = 50$, $P_c = 1$, $P_m = 0,1$, maxevals**Ensure:** Mejor solución encontrada y su fitness

```

1: for  $i \leftarrow 0$  hasta  $M - 1$  do
2:    $\text{pop}[i] \leftarrow \text{CREATE\_SOLUTION}()$ 
3:    $\text{fits}[i] \leftarrow \text{FITNESS}(\text{pop}[i]); \text{evals} += 1$ 
4:  $(\text{bestSol}, \text{bestFit}) \leftarrow$  individuo de menor fitness en pop
5:
6: while  $\text{evals} < \text{maxevals}$  do
7:    $p_1 \leftarrow \text{TOURNAMENT\_SELECT}(\text{pop}, \text{fits})$  ▷ Selección
8:    $p_2 \leftarrow \text{TOURNAMENT\_SELECT}(\text{pop}, \text{fits})$ 
9:
10:   $(c1, c2) \leftarrow \text{DOCROSSOVER}(\text{pop}[p_1], \text{pop}[p_2])$  ▷ Cruce
11:
12:  if  $\text{RealAleatorio}(0, 1) < P_m$  then  $\text{MUTATE}(c1, k)$  ▷ Mutación
13:  if  $\text{RealAleatorio}(0, 1) < P_m$  then  $\text{MUTATE}(c2, k)$ 
14:
15:  if  $\neg \text{ISVALID}(c1)$  then  $\text{FIX}(c1)$  ▷ Reparación
16:  if  $\neg \text{ISVALID}(c2)$  then  $\text{FIX}(c2)$ 
17:
18:  if  $\text{evals} \geq \text{maxevals}$  then break ▷ Evaluación
19:   $f_1 \leftarrow \text{FITNESS}(c1); \text{evals} += 1$ 
20:   $f_2 \leftarrow f_1$ 
21:  if  $\text{evals} < \text{maxevals}$  then
22:     $f_2 \leftarrow \text{FITNESS}(c2); \text{evals} += 1$ 
23:
24:   $(w_1, w_2) \leftarrow$  índices de los 2 peores de pop ▷ Reemplazo
25:  if  $f_1 < \text{fits}[w_1]$  then
26:     $\text{pop}[w_1] \leftarrow c1; \text{fits}[w_1] \leftarrow f_1$ 
27:  else if  $f_1 < \text{fits}[w_2]$  then
28:     $\text{pop}[w_2] \leftarrow c1; \text{fits}[w_2] \leftarrow f_1$ 
29:  if  $f_2 < \text{fits}[w_1]$  then
30:     $\text{pop}[w_1] \leftarrow c2; \text{fits}[w_1] \leftarrow f_2$ 
31:  else if  $f_2 < \text{fits}[w_2]$  then
32:     $\text{pop}[w_2] \leftarrow c2; \text{fits}[w_2] \leftarrow f_2$ 
33:
34:  if  $\min(\text{fits}) < \text{bestFit}$  then ▷ Actualizar mejor global
35:     $(\text{bestSol}, \text{bestFit}) \leftarrow$  mejor individuo de pop
36: return  $\text{bestSol}, \text{bestFit}$ 

```

3.6. Algoritmos Meméticos (AM)

Los algoritmos meméticos hibridan el AGG con una Búsqueda Local Suave (BLS) aplicada cada 10 generaciones sobre un subconjunto de cromosomas. Como base se ha seleccionado **AGG-UN**, cuya elección se justifica en el análisis de resultados (Sección 5.3.4).

Para este algoritmo la población será $M = 50$, la probabilidad de cruce será de $P_c = 0,8$, la probabilidad de mutación será de $P_m = 0,1$. Cuando la BLS no produce cambios en el cromosoma decimos que falla. El contador de fallos está diseñado para evitar que se desperdicien evaluaciones de la función objetivo en cromosomas de mucha calidad. Permitiremos como mucho ϵ fallos en la BLS, siendo $\epsilon = 0,1 \cdot N$. El criterio de parada del AM consistirá en realizar 100000 evaluaciones de la función objetivo, incluidas por supuesto las de la BLS. Se aplicará la BL cada 10 generaciones del algoritmo, y cada vez que se aplique la BL se hará durante 100 evaluaciones. Por tanto, $maxevals = 100000$, $BLS_{maxevals} = 100$, $\xi = \lfloor 0,1 \cdot n \rfloor$, y el período BLS = 10 generaciones.

3.6.1. Búsqueda Local Suave (BLS)

La BLS recorre las posiciones del cromosoma en orden aleatorio y, para cada posición, evalúa todos los k valores posibles y se queda con el mejor. Se detiene si ξ posiciones consecutivas no mejoran, si se han visitado todas las posiciones, o si se agota el presupuesto de evaluaciones.

Algorithm 18 Búsqueda Local Suave – BLS(S , bestfit, maxevals, ϵ)

Require: Solución S , fitness bestfit, maxevals, $\epsilon = \lfloor 0,1 \cdot n \rfloor$

Ensure: Solución mejorada, su fitness, evaluaciones usadas

```

1: RSI  $\leftarrow$  ORDENALEATORIO( $[0, \dots, n - 1]$ )
2: fallos  $\leftarrow 0$ ;  $i \leftarrow 0$ ; newsol  $\leftarrow S$ 
3: while fallos <  $\epsilon$  y  $i < n$  y evals < maxevals do
4:    $p \leftarrow$  RSI[ $i$ ]; oldvalue  $\leftarrow S[p]$ 
5:   for val  $\leftarrow 0$  hasta  $k - 1$  mientras evals < maxevals do
6:     if val = newsol[ $p$ ] then continuar
7:     cnt  $\leftarrow |\{j : \text{newsol}[j] = \text{newsol}[p]\}|$ 
8:     if cnt  $\leq 1$  then continuar
9:     newsol[ $p$ ]  $\leftarrow$  val
10:    fit  $\leftarrow$  FITNESS(newsol); evals += 1
11:    if fit < bestfit then
12:       $S \leftarrow$  newsol; bestfit  $\leftarrow$  fit
13:    else
14:      newsol[ $p$ ]  $\leftarrow S[p]$ 
15:    if newsol[ $p$ ] = oldvalue then fallos += 1
16:     $i += 1$ 
17: return  $S$ , bestfit, evals

```

3.6.2. Variantes del Algoritmo Memético

Las tres variantes difieren únicamente en los cromosomas sobre los que se aplica la BLS cada 10 generaciones:

- **AM-All:** Se aplica la BLS a todos los cromosomas de la población ($M = 50$).
- **AM-Rand:** Se aplica la BLS a cada cromosoma con probabilidad $p_{LS} = 0,1$ de forma independiente.
- **AM-Best:** Se aplica la BLS a los $\lfloor 0,1 \cdot M \rfloor$ mejores cromosomas de la población.

El esquema base del AM es idéntico al AGG, añadiendo el siguiente bloque al final de cada generación:

Algorithm 19 Aplicación de BLS en AM – cada 10 generaciones

```

1: if generación mód10 = 0 y evals < maxevals then
2:   objetivos  $\leftarrow$  SELECTTARGETS(fits,  $M$ )  $\triangleright$  según variante: ALL / RAND / BEST
3:   for cada índice  $idx$  en objetivos do
4:     if evals  $\geq$  maxevals then salir
5:     blsEvals  $\leftarrow$  mín(100, maxevals – evals)
6:     (pop[ $idx$ ], fits[ $idx$ ], used)  $\leftarrow$  BLS(pop[ $idx$ ], fits[ $idx$ ], blsEvals,  $\xi$ )
7:     evals += used
8:     Actualizar bestSol, bestFit si es necesario
  
```

4. Estructura del Código y Manual de Usuario

4.1. Estructura de Directorios

El proyecto se organiza en los siguientes directorios y ficheros:

- **main.cpp**: Punto de entrada. Carga los seis casos del problema, ejecuta los diez algoritmos (Greedy, Random, BL, AGG-UN, AGG-SF, AGE-UN, AGE-SF, AM-All, AM-Rand, AM-Best) con las semillas especificadas y muestra los resultados en formato tabular.
- **common/**: Framework base independiente del problema.
 - **mh.h**: Interfaz genérica **MH<T>** que deben implementar todos los algoritmos, y estructura **ResultMH<T>** que encapsula solución, fitness y número de evaluaciones.
 - **problem.h**: Interfaz genérica **Problem<T>** con los métodos **fitness**, **createSolution**, **isValid** y **fix**.
 - **solution.h**: Definición de los tipos **tSolution<T>** y **tFitness**.
 - **random.hpp**: Generador pseudoaleatorio de calidad basado en Mersenne Twister. Proporciona **Random::seed()**, **Random::get()** y **Random::shuffle()**.
 - **util.h**: Utilidades auxiliares.
- **inc/**: Cabeceras específicas del PAR.
 - **ppar.h**: Clase **PARProblem**. Carga el dataset y la matriz de restricciones, construye las listas ML y CL, calcula λ , e implementa las funciones **fitness**, **deviation**, **infeasibility**, **createSolution**, **isValid** y **fix**.
 - **genetic.h**: Operadores genéticos compartidos (**tournamentSelect**, **crossoverUniform**, **crossoverFixedSegment**, **mutate**) y declaraciones de las clases AGG y AGE.
 - **memetic.h**: Función **bls** y declaración de la clase AM.
 - **greedy_copkm.h**: Cabecera del algoritmo COPKM.
 - **localsearch.h**: Cabecera de la Búsqueda Local.

- `randomsearch.h`: Cabecera de la Búsqueda Aleatoria.
- `src/`: Implementaciones de los algoritmos.
 - `genetic_agg.cpp`: Implementación de `AGG::optimize`.
 - `genetic_age.cpp`: Implementación de `AGE::optimize`.
 - `memetic.cpp`: Implementación de `AM::optimize` y `AM::selectTargets`.
 - `greedy_copkm.cpp`: Implementación del algoritmo Greedy COPKM.
 - `localsearch.cpp`: Implementación de la Búsqueda Local.
 - `randomsearch.cpp`: Implementación de la Búsqueda Aleatoria.
- `data/`: Directorio con los ficheros de datos (`.dat`) y de restricciones de los seis casos del problema.
- `CMakeLists.txt`: Fichero de configuración del sistema de construcción CMake.

4.2. Compilación

El proyecto utiliza **CMake** (versión $\geq 3,10$) con estándar C++17. Para compilar el proyecto se deben ejecutar los siguientes comandos desde el directorio raíz:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel 4
```

El ejecutable resultante se genera en `build/main`.

4.3. Ejecución

La sintaxis de ejecución es la siguiente:

```
./build/main [--verbose] [semilla1 semilla2 ... semilla10]
```

Los parámetros son opcionales:

- `--verbose` o `-v`: muestra adicionalmente, para cada caso y algoritmo, el vector de asignación de la última ejecución, los tamaños de cada cluster, el número de restricciones ML y CL violadas y el desglose completo de la función objetivo.
- `semilla1 ... semilla10`: valores enteros que inicializan el generador pseudoaleatorio en cada una de las 10 ejecuciones. Si no se especifican, se usan por defecto los valores $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$.

4.4. Ejemplos de Ejecución

```
# Ejecución con 10 semillas (por defecto)
./build/main 1 2 3 4 5 6 7 8 9 10
```

```
# Ejecución con verbose (muestra solución detallada por algoritmo)
./build/main --verbose 1 2 3 4 5 6 7 8 9 10
```

5. Experimentos y Análisis de Resultados

5.1. Configuración Experimental

El estudio experimental involucra tres conjuntos de datos reales: Zoo (101 instancias, 16 atributos, $k = 7$), Glass (214 instancias, 9 atributos, $k = 7$) y Bupa (345 instancias, 5 atributos, $k = 16$). A partir de cada conjunto de datos se han obtenido 2 instancias distintas generando 2 conjuntos de restricciones, correspondientes al 15 % y 30 % del total de restricciones posibles, dando lugar a un total de 6 instancias del PAR. Los parámetros λ resultantes para cada caso se recogen en la Tabla 1.

Se han ejecutado los diez algoritmos sobre los seis casos del problema (Zoo 15 %, Zoo 30 %, Glass 15 %, Glass 30 %, Bupa 15 %, Bupa 30 %) con las semillas $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$. Los resultados que se muestran son la media de las 10 ejecuciones.

Tabla 1: Características de los conjuntos de datos

Dataset	n	d	k	Restricciones
Zoo 15 %	101	16	7	766
Zoo 30 %	101	16	7	1531
Glass 15 %	214	9	7	1621
Glass 30 %	214	9	7	3243
Bupa 15 %	345	5	16	2982
Bupa 30 %	345	5	16	5964

Cabe recordar la nomenclatura: n instancias, d atributos por instancia, k clusters y R restricciones.

5.2. Resultados Obtenidos

Tabla 2: Resultados para Zoo con 15 % de restricciones

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Greedy	1.945	1.134	0.220	1	0.034
Random	1.931	1.578	0.096	100000	0.494
BL	1.075	0.921	0.042	6531	0.013
AGG-UN	1.095	0.941	0.042	100000	0.240
AGG-SF	1.099	0.941	0.043	100000	0.252
AGE-UN	1.060	0.917	0.039	100000	0.247
AGE-SF	1.060	0.925	0.037	100000	0.263
AM-All	1.085	0.926	0.043	100000	0.174
AM-Rand	1.076	0.918	0.043	100000	0.209
AM-Best	1.055	0.891	0.044	100000	0.227

Tabla 3: Resultados para Zoo con 30 % de restricciones

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Greedy	2.917	0.642	0.618	1	0.069
Random	1.976	1.493	0.131	100000	0.510
BL	1.206	0.871	0.091	5640	0.012
AGG-UN	1.192	0.849	0.093	100000	0.260
AGG-SF	1.192	0.848	0.093	100000	0.268
AGE-UN	1.219	0.892	0.089	100000	0.262
AGE-SF	1.198	0.861	0.091	100000	0.280
AM-All	1.195	0.852	0.093	100000	0.192
AM-Rand	1.207	0.870	0.091	100000	0.228
AM-Best	1.203	0.862	0.092	100000	0.230

Tabla 4: Resultados para Glass con 15 % de restricciones

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Greedy	1.057	0.234	0.386	1	0.291
Random	0.681	0.424	0.121	100000	0.986
BL	0.404	0.302	0.048	14744	0.052
AGG-UN	0.412	0.303	0.051	100000	0.433
AGG-SF	0.411	0.301	0.052	100000	0.457
AGE-UN	0.413	0.308	0.049	100000	0.444
AGE-SF	0.413	0.310	0.048	100000	0.472
AM-All	0.408	0.299	0.051	100000	0.317
AM-Rand	0.404	0.302	0.048	100000	0.367
AM-Best	0.405	0.304	0.047	100000	0.366

Tabla 5: Resultados para Glass con 30 % de restricciones

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Greedy	1.247	0.190	0.496	1	0.586
Random	0.697	0.423	0.129	100000	1.121
BL	0.467	0.286	0.085	20048	0.094
AGG-UN	0.477	0.292	0.087	100000	0.551
AGG-SF	0.473	0.288	0.087	100000	0.570
AGE-UN	0.472	0.289	0.086	100000	0.566
AGE-SF	0.473	0.286	0.088	100000	0.588
AM-All	0.471	0.288	0.086	100000	0.432
AM-Rand	0.468	0.285	0.086	100000	0.485
AM-Best	0.469	0.286	0.086	100000	0.485

Tabla 6: Resultados para Bupa con 15 % de restricciones

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Greedy	0.445	0.153	0.194	1	2.928
Random	0.329	0.248	0.054	100000	1.800
BL	0.170	0.143	0.018	91170	0.781
AGG-UN	0.198	0.164	0.023	100000	0.972
AGG-SF	0.200	0.167	0.022	100000	0.996
AGE-UN	0.195	0.163	0.021	100000	0.996
AGE-SF	0.196	0.163	0.022	100000	1.025
AM-All	0.202	0.173	0.019	100000	0.801
AM-Rand	0.179	0.150	0.019	100000	0.878
AM-Best	0.177	0.148	0.019	100000	0.877

Tabla 7: Resultados para Bupa con 30 % de restricciones

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (s)
Greedy	0.525	0.112	0.274	1	5.950
Random	0.333	0.247	0.057	100000	2.172
BL	0.198	0.144	0.036	85020	1.063
AGG-UN	0.221	0.160	0.040	100000	1.353
AGG-SF	0.220	0.159	0.041	100000	1.376
AGE-UN	0.217	0.157	0.040	100000	1.359
AGE-SF	0.218	0.159	0.039	100000	1.392
AM-All	0.221	0.163	0.039	100000	1.178
AM-Rand	0.204	0.149	0.036	100000	1.257
AM-Best	0.202	0.147	0.037	100000	1.254

5.3. Análisis de Resultados

Los resultados de esta práctica se comparan con los algoritmos implementados en la Práctica 1 (Greedy, Random y BL), que sirven de referencia para evaluar el beneficio aportado por las técnicas basadas en poblaciones. El objetivo es determinar en qué medida los Algoritmos Genéticos y Meméticos mejoran sobre las metaheurísticas de trayectoria simple ya conocidas, y bajo qué condiciones cada enfoque resulta más adecuado. Las Figuras 1 y 2 muestran la distribución del fitness para los algoritmos BL, AGG, AGE y AM sobre los seis casos del problema, excluyendo Greedy y Random cuya escala haría imperceptibles las diferencias entre el resto de algoritmos.

5.3.1. Greedy y Random como referencia

El Greedy obtiene los peores resultados en todos los datasets. Su estrategia de asignación secuencial prioriza en cada paso minimizar el número de restricciones violadas usando únicamente la información de las instancias ya asignadas. Esto lleva a decisiones irrecuperables en las primeras etapas. Para evitar violar restricciones inmediatas, concentra demasiadas instancias en pocos clusters, generando particiones muy desequilibradas con alta distancia intra-cluster. Este efecto se agrava con más restricciones, lo que explica el colapso en Zoo 30 % (fitness 2,917, casi el doble que en Zoo 15 %), como es un dataset

pequeño cada restricción adicional afecta a una proporción mayor del total de asignaciones posibles. En Glass y Bupa el deterioro es más moderado porque el mayor número de instancias diluye el efecto de cada restricción individual.

Random mejora al Greedy en todos los casos, lo que confirma que la estrategia greedy produce particiones peores que las puramente aleatorias. Además, Random muestra escasa diversidad entre ejecuciones. Al asignar instancias aleatoriamente, la mayoría de soluciones distribuyen las instancias de forma aproximadamente uniforme entre clusters, convergiendo a valores similares de $\bar{C}$ independientemente de la semilla, y violando un número proporcional de restricciones en todos los casos.

5.3.2. Búsqueda Local como el algoritmo más fuerte

La Búsqueda Local obtiene el mejor fitness en cuatro de los seis casos (Glass 15 %, Glass 30 %, Bupa 15 %, Bupa 30 %), y es competitiva en los dos restantes. Esto puede sorprender dado que los AGs y AMs usan el mismo presupuesto de evaluaciones, pero se explica por cómo cada algoritmo lo distribuye. BL invierte sus evaluaciones íntegramente en mejorar una única solución mediante movimientos de primer mejor, mientras que los algoritmos poblacionales reparten el mismo presupuesto entre la evaluación de 50 cromosomas por generación. La mayoría de estos cromosomas son soluciones mediocres producidas por cruce y mutación sin garantía de mejorar, ya que combinar dos buenas soluciones en PAR no garantiza producir otra buena. Esto es así porque las etiquetas de cluster no tienen un significado posicional, y por tanto el cruce puede romper agrupaciones útiles de los padres y generar descendientes con muchas restricciones violadas que luego deben repararse.

La baja variabilidad de BL visible en los boxplots, especialmente en Bupa donde la caja es casi plana, confirma que el algoritmo converge de forma muy consistente independientemente de la solución inicial. En Zoo la variabilidad es mayor porque el espacio de búsqueda es más pequeño y las diferencias entre óptimos locales son más apreciables en términos relativos.

La única excepción es Zoo, donde AGE-UN y AM-Best superan a BL. En un dataset de solo 101 instancias el espacio de búsqueda es suficientemente reducido para que la exploración de múltiples soluciones simultáneas aporte valor, permitiendo a los algoritmos poblacionales escapar de los óptimos locales en los que BL puede quedar atrapado.

Que BL supere a los algoritmos meméticos en Glass y Bupa es aparentemente contrario a la teoría, que predice que un algoritmo memético debería ser al menos tan bueno como la búsqueda local que incorpora. Sin embargo, esta aparente anomalía se explica por el diseño específico de esta práctica. La BLS utilizada dentro de los AMs está deliberadamente limitada a 100 evaluaciones por aplicación y visita cada posición solo una vez, siendo mucho más superficial que la BL completa. Además, el presupuesto de 100.000 evaluaciones se reparte entre la evolución del AG y las aplicaciones de BLS, por lo que ninguna solución individual recibe tanta atención como la que BL dedica a su única solución. Si los AMs utilizasen la BL completa en lugar de la BLS, se podría esperar que superaran a la BL, al combinar exploración poblacional con explotación profunda.

5.3.3. AGG vs AGE: influencia del esquema de evolución

AGE supera a AGG en cuatro de los seis casos, con diferencias más pronunciadas en Bupa. La causa está en el esquema de reemplazo. Mientras que el AGG sustituye la población completa cada generación, lo que implica que en cada iteración los 50 offspring reemplazan a los 50 padres independientemente de su calidad relativa (salvo el elitismo).

Esto genera una presión selectiva elevada que puede reducir la diversidad poblacional rápidamente cuando los mejores individuos dominan la selección por torneo. El AGE, al reemplazar solo dos individuos por iteración y únicamente si los hijos son mejores que los peores de la población, mantiene más diversidad durante más generaciones y por tanto permite una exploración más prolongada del espacio de búsqueda.

En Bupa, con $n = 345$ y $k = 16$, el espacio de búsqueda es mucho mayor y la diversidad es más relevante para obtener buenas soluciones, lo que explica que la ventaja del AGE sea más pronunciada allí. En Zoo 30 %, donde AGG-UN (1,192) supera a AGE-UN (1,219), el espacio más pequeño hace que la mayor presión selectiva del AGG no sea perjudicial.

La mayor variabilidad de los AGs respecto a BL, visible en los boxplots con cajas más anchas y bigotes más largos, se explica porque distintas semillas generan poblaciones iniciales diferentes que pueden evolucionar hacia regiones muy distintas del espacio de búsqueda.

5.3.4. AGG-UN vs AGG-SF y AGE-UN vs AGE-SF: influencia del operador de cruce

Las diferencias entre cruce uniforme y segmento fijo son mínimas en todos los casos, con cajas prácticamente solapadas en los boxplots. El cruce de segmento fijo está diseñado para preservar bloques de genes contiguos que funcionan bien juntos, lo cual tiene sentido en problemas donde la posición en el vector tiene significado geométrico (como el TSP). Sin embargo, en PAR, el orden de las instancias en el vector es completamente arbitrario, por lo que preservar un segmento contiguo de etiquetas no garantiza heredar propiedades útiles del padre. Por este motivo, ambos operadores producen resultados equivalentes.

Dado que las diferencias son mínimas y AGG-UN presenta una ligera ventaja global, se ha seleccionado como base para los algoritmos meméticos.

5.3.5. Algoritmos Meméticos: balance exploración-explotación

Los meméticos mejoran sobre los AGs puros en casi todos los casos gracias a la incorporación de la BLS, que refina localmente las soluciones. Sin embargo, no superan a BL en Glass ni en Bupa, porque la BLS aplicada en los meméticos está deliberadamente limitada a 100 evaluaciones por aplicación y visita cada posición solo una vez, siendo mucho más superficial que la BL completa de la Práctica 1.

AM-All es el peor de los tres meméticos en cinco de los seis casos. Al aplicar BLS a los 50 cromosomas cada 10 generaciones, se consume $50 \cdot 100 = 5,000$ evaluaciones por aplicación en refinamiento local. Esto deja menos presupuesto para la evolución del AG base, reduciendo el número efectivo de generaciones y por tanto la exploración del espacio de búsqueda. El resultado, visible en los boxplots, son cajas estrechas (convergencia consistente) pero con medianas más altas que AM-Best y AM-Rand, porque la población apenas tiene tiempo de evolucionar antes de que se aplique de nuevo la BLS.

AM-Best, al aplicar BLS solo a los 5 mejores cromosomas por aplicación ($5 \times 100 = 500$ evaluaciones), mantiene más presupuesto para la evolución y concentra el refinamiento en las regiones más prometedoras. AM-Rand, con una selección aleatoria de cromosomas con probabilidad 0,1, tiene un coste similar a AM-Best pero distribuye el refinamiento de forma menos sistemática, lo que le da cajas más anchas en los boxplots pero medianas competitivas.

5.3.6. Efecto del nivel de restricciones

Pasar del 15 % al 30 % de restricciones empeora el fitness en todos los algoritmos. El ranking entre algoritmos se mantiene estable en Glass y Bupa, pero en Zoo cambia puesto que AGE-UN supera a AGG-UN en 15 % pero queda por debajo en 30 %. Esto sugiere que con más restricciones la presión selectiva más alta del AGG puede ser ventajosa en datasets pequeños, donde converger rápido hacia soluciones con pocas violaciones es prioritario.

Cabe señalar que λ disminuye al aumentar el número de restricciones: de 0,005546 a 0,002577 en Zoo, de 0,000662 a 0,000320 en Glass, y de 0,000176 a 0,000086 en Bupa. Esta reducción atenúa parcialmente el incremento de la penalización, lo que explica por qué el deterioro en fitness no es proporcional al doble de restricciones añadidas.

“`latex`”

5.3.7. Conclusiones

En términos de calidad de la solución, BL es el algoritmo más efectivo en cuatro de los seis casos, obteniendo el mejor fitness en Glass y Bupa a ambos niveles de restricciones. En Zoo, donde el espacio de búsqueda es más pequeño, AM-Best obtiene el mejor resultado global del experimento (1,055 en Zoo 15 %), siendo la única configuración que supera consistentemente a BL. Los AGs puros quedan siempre por debajo de BL y de los AMs, lo que indica que la exploración poblacional sin refinamiento local no es suficiente para este problema.

En términos de tiempo de ejecución, BL es también el algoritmo más rápido en todos los casos sin excepción, con tiempos entre 0,020s y 1,719s. Esto es especialmente llamativo en Bupa, donde BL resuelve el problema en 1,291s (15 %) y 1,719s (30 %), mientras que los AGs tardan entre 1,588s y 2,270s usando el mismo presupuesto de evaluaciones. La razón es que BL termina antes de agotar las 100.000 evaluaciones en la mayoría de casos, mientras que los AGs siempre las consumen íntegramente.

Los AMs son más rápidos que los AGs puros a pesar de incorporar BLS, porque la BLS reduce el número de generaciones que el AG base puede completar dentro del presupuesto, acortando el tiempo total. Sin embargo, esta reducción de tiempo no se traduce en mejor calidad respecto a los AGs en la mayoría de casos, salvo en Zoo donde AM-Best combina mejor calidad y tiempo razonable.

En conjunto, si se prioriza tanto la calidad como el tiempo de ejecución, BL es la opción más equilibrada para Glass y Bupa, y AM-Best para Zoo. Los AGs puros no ofrecen ventaja ni en calidad ni en tiempo en ningún caso. La superioridad de BL sobre los AMs no contradice la teoría de los algoritmos meméticos, sino que refleja el diseño de baja intensidad de la BLS en esta práctica: en cuanto el espacio de búsqueda es suficientemente complejo para que la exploración poblacional aporte valor, como ocurre en Zoo, los meméticos demuestran su ventaja teórica.

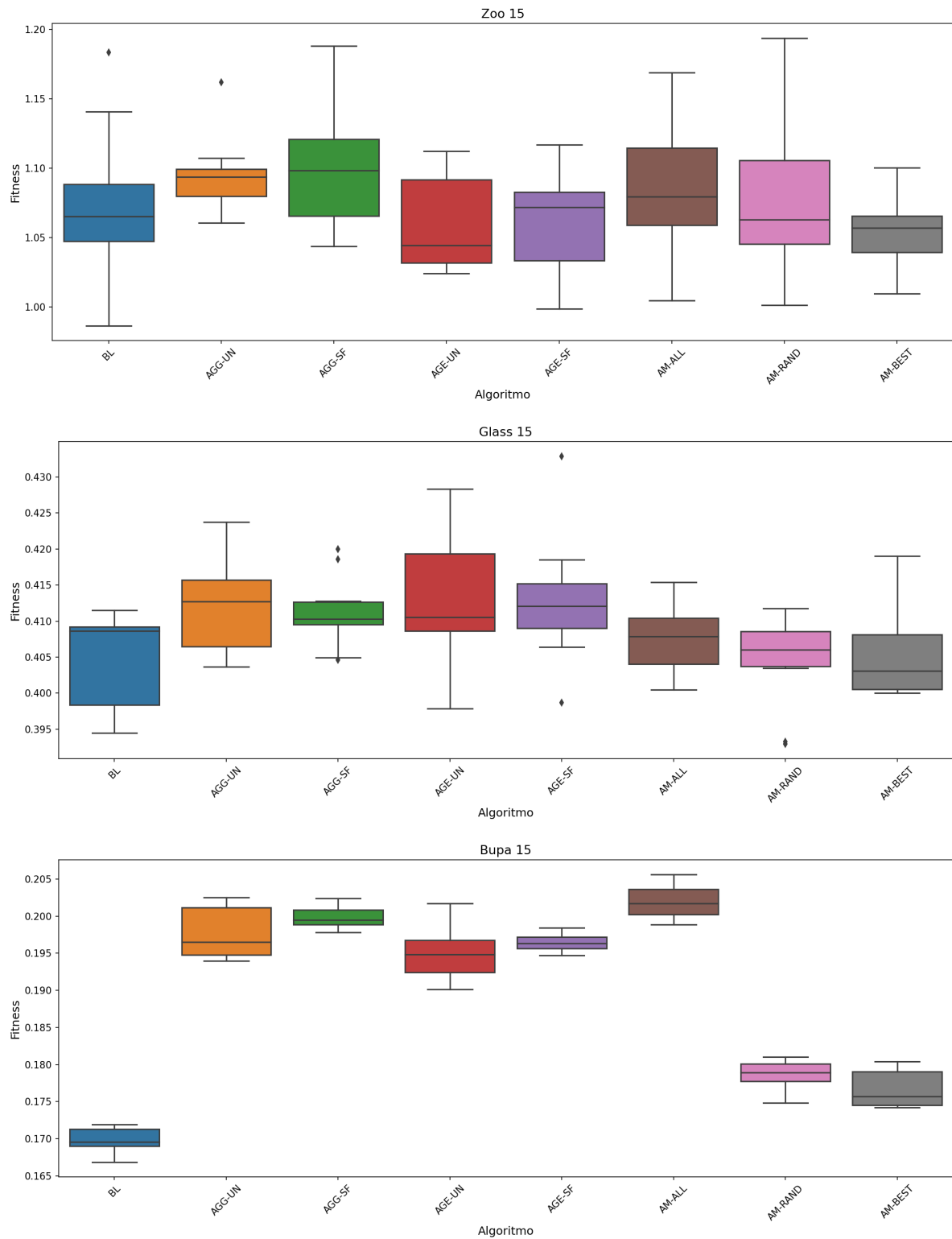


Figura 1: Distribución del fitness para BL, AGGs, AGEs y AMs. Cada fila corresponde a un dataset (Zoo, Glass, Bupa de arriba a abajo) con 15 % de restricciones

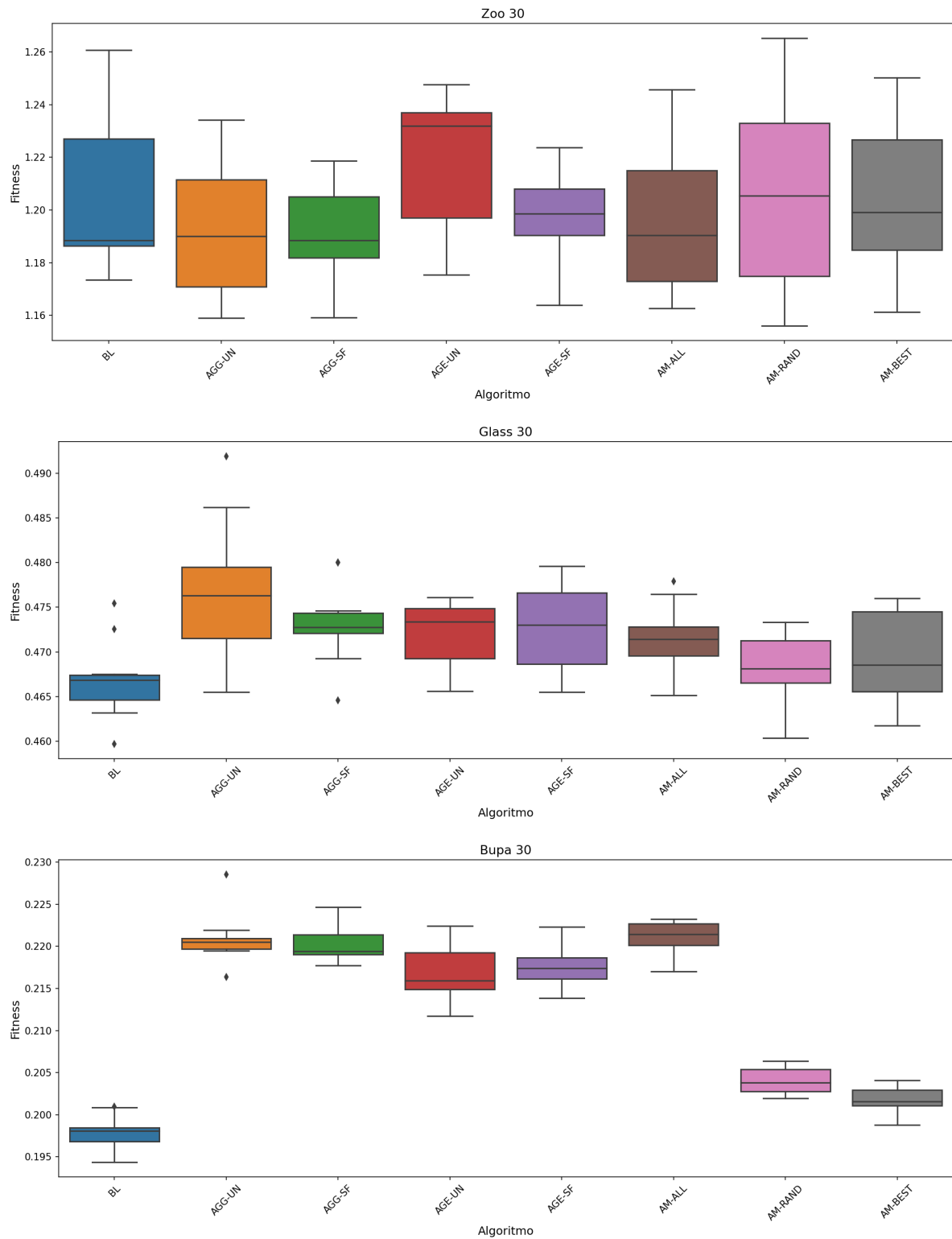


Figura 2: Distribución del fitness para BL, AGGs, AGEs y AMs. Cada fila corresponde a un dataset (Zoo, Glass, Bupa de arriba a abajo) con 30 % de restricciones